
Algorithm 1 VerifyCasting(Instance, Certificate)

Require: Instance containing n (roles), s (scenes), k (actors), $AllowedActors[1..n]$, and $SceneRoles[1..s]$.

Require: Certificate array $Cast[1..n]$.

Ensure: **ACCEPT** if the certificate is valid, **REJECT** otherwise.

▷ 1. Verify that the certificate covers exactly n roles

```
1: if length( $Cast$ )  $\neq n$  then
2:   return REJECT
3: end if
```

```
4:  $p1\_participates \leftarrow \text{FALSE}$ 
5:  $p2\_participates \leftarrow \text{FALSE}$ 
```

▷ 2. Verify Type 1 constraints and track actor participation

```
6: for  $i = 1 \dots n$  do
7:    $actor \leftarrow Cast[i]$ 
8:   if  $actor \notin AllowedActors[i]$  then
9:     return REJECT
10:  end if
11:  if  $actor == 1$  then
12:     $p1\_participates \leftarrow \text{TRUE}$ 
13:  end if
14:  if  $actor == 2$  then
15:     $p2\_participates \leftarrow \text{TRUE}$ 
16:  end if
17: end for
```

▷ 3. Verify both p_1 and p_2 participate in the play

```
18: if not ( $p1\_participates$  and  $p2\_participates$ ) then
19:   return REJECT
20: end if
```

▷ 4. Verify Type 2 constraints and scene exclusions

```
21: for  $j = 1 \dots s$  do
22:    $p1\_in\_scene \leftarrow \text{FALSE}$ 
23:    $p2\_in\_scene \leftarrow \text{FALSE}$ 
24:    $ActorsInCurrentScene \leftarrow \emptyset$ 
```

```
25:   for  $role \in SceneRoles[j]$  do
26:      $actor \leftarrow Cast[role]$ 
27:     if  $actor \in ActorsInCurrentScene$  then
28:       return REJECT   ▷ One actor cannot play multiple roles in a
scene
```

```
29:     end if
30:      $ActorsInCurrentScene.add(actor)$ 
```

```
31:     if  $actor == 1$  then
32:        $p1\_in\_scene \leftarrow \text{TRUE}$ 
33:     end if
34:     if  $actor == 2$  then
35:        $p2\_in\_scene \leftarrow \text{TRUE}$ 
36:     end if
37:   end for
```

```
38:   if  $p1\_in\_scene$  and  $p2\_in\_scene$  then
39:     return REJECT   ▷  $p_1$  and  $p_2$  cannot be in the same scene
40:   end if
41: end for
```

```
42: return ACCEPT
```
